

Proiect 1 – Client-Server

Obiectiv:

- Implementarea unei aplicatii client-server
- Folosirea executiei concurente prin apeluri asincrone.
- Folosirea mecanismelor: future/promises si thread_pool.
- Analiza imbunatatirii performantei executiei unei aplicatii (de tip business) prin programare concurenta.

Clinica medicala

O statiune balneara ofera diverse spatii pentru diverse tratamente (acces piscina, masaj, impachetari, etc.) Din cauza existentei unei cereri mari se solicita plata la momentul programarii, existand totusi posibilitatea anularii, caz in care se face si returnarea banilor.

Exista n locatii ($S_1, S_2, \dots, S_n$) la care se pot face programari, fiecare locatie oferind acelasi set de tratamente ($T(1), T(2), \dots, T(m)$). Locatiile sunt deschise intre orele 10:00-18:00 si fiecare tratament are asociat un cost $C(j)$, o anumita durata de realizare $T_{\text{delta}}(j)$ si un numar maxim de pacienti care pot fi tratati in acelasi timp $N(i,j)$ $0 < i \leq n$; $0 < j \leq m$.

La prima iteratie de dezvoltare aplicatia de programari nu ofera interfata grafica prin care sa se vizualizeze situatia la zi a programarilor. Un client va incerca sa isi faca o programare in ziua si in intervalul dorit si va primi raspuns „programare reusita” sau „programare nereusita” in functie de disponibilitate.

O programare care este posibila (nu exista suprapuneri) se face mai intai cu status „rezervare” si dupa ce se inregistreaza plata se schimba statusul in „platita”. Daca plata nu se realizeaza in timp de T_{plata} atunci programarea se sterge.

Dupa mesajul de programare reusita se va cere plata si apoi clientul trimite cererea de plata furnizand datele pentru plata.

Pentru planificare se adauga o inregistrare cu urmatoarele informatii: (id, nume, cnp, data, locatie_tratament, tip_tratament, data_tratament, ora_tratament).

Pentru plata se adauga o inregistrare cu urmatoarele informatii: (id, data, cnp, suma).

Pentru anulare se sterge inregistrarea platii corespunzatoare planificarii si pentru returnare bani se adauga o inregistrare cu urmatoarele informatii: (id, data, cnp, -suma).

Observatie: Salvarea poate fi facuta intr-o baza de date sau in fisiere de tip text.

Periodic sistemul (2 cazuri testare: 5, 10 secunde) face o verificare a programarilor facute si a incasarilor corespunzatoare prin verificarea corespondentei corecte intre numarul programarile facute si sumele incasate dar si calcularea soldului total. Prin aceasta actiune se verifica si faptul ca nu sunt suprapuneri in planificari (numarul total de clienti programati la locatie i , pentru tratamentul $T(j)$ la un anumit timp nu este mai mare decat numarul maxim $N(i,j)$ admis).

Atentie: este important sa se asigure faptul ca verificarile nu se pot face la un moment in care o anulare nu este complet realizata (actualizare planificare si returnare bani), dar se pot face intre o cerere de programare confirmata si plata corespunzatoare ei. Verificarea trebuie sa semnalizeze programarile neplatite.

Pentru simplificare se va considera ca toate cererile se fac pentru aceeasi zi.

Serverul va putea folosi maxim p threaduri pentru rezolvarea cererilor.

Pentru fiecare cerere de programare se va folosi o constructie de tip future.

Pentru testare se va considera ca fiecare client initiaza/creeaza la interval de (2 sec) o noua cerere de programare folosind date generate aleatoriu (locatie, tip_tratament, ora) si se primeste de la server o notificare – „programare reusita” sau „programare nereusita”. Nu este necesara interfata grafica!

Pentru verificare se cere salvarea pe suport extern (fisier text) a rezultatelor operatiilor de verificare executate periodic:

Pentru fiecare locatie se salveaza:

Ora verificării: (locatie, sold_total_locatie) + lista programarilor neplatite+ eventuale suprapuneri

Serverul se inchide dupa un interval de timp precizat si notifica clientii activi referitor la inchidere.

Limbajul de implementare: la alegere

Testare:

p=10

Numar clienti=10

Tipuri de tratament n=5, m=5

Costuri tratamente: 50, 20, 40, 100, 30

Durate tratamente (minute): 120, 20, 30, 60, 30

Numar maxim de clienti care se pot face simultan un tratament:

$N(1,1)=3$, $N(1,2)=1$, $N(1,3)=1$, $N(1,4)=2$, $N(1,5)=1$

$N(i,j)=N(1,j)*(i-1)$ pentru toti $i>1$ si $0<j\leq 5$

Serverul lucreaza 3 minute.

Cerinte de livrare/prezentare

Se incarca in assignment!

- Cod server + cod client + script/test .
- Fişiere rezultate din rulări (5s şi 10s).
- Documentaţie (README): cum rulezi, parametri, mediu de lucru
- Raport scurt (2–4 pagini):
arhitectură, decizii de sincronizare, cum ai folosit futures, thread pool, analiza performanţei (timpi medii de răspuns, throughput)

Prezentarea implica faptul ca

orice detaliu de proiectare si implementare poate fi explicat!